Practical - 01

AIM : Program to implement Lexical Analyser.

Program:

```
#include    <stdio.h>
#include    <ctype.h>
#include    <string.h>
#include    <stdlib.h>
void keywI(char *p);
int i = 0, id = 0, kw = 0, num = 0, op = 0, sp = 0, ar = 0, count = 0, new_line = 0;
char keys[32][10] = {"auto", "break", "case", "char", "const", "continue", "default", "do",
"double", "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
"return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef", "union",
"unsigned", "void", "volatile", "while"};
int main() {
char ch, str[25], seps[20] = "\t\n,(){}[]#\"<>", oper[] = "!%^&*-+=~|<>?";
int j;
FILE *f1;
f1 = fopen("Laxcode.txt", "r");
if (f1 == NULL) {
printf("file not found\n");
exit(0);
}
 while ((ch = fgetc(f1)) != EOF) {
 for (j = 0; j < strlen(oper); j++) {
            if (ch == oper[j]) {
            printf("%c is an operator\n", ch);
            op++;
            count++;
            str[i] = '\0';
            keywI(str);
            i = -1;
            break;
            }
}
if (ch == '\n') {
            new_line++;
}
```

```
for (j = 0; j < strlen(seps); j++) {
        if (i == -1)
        break;
        if (ch == seps[j]) {
        if (ch == '#') {
                while (ch != '>') {
                printf("%c", ch);ch = fgetc(f1);
                }
                printf("%c is a header file\n", ch);
                i = -1;
                break;
        }
        if (ch == ' ') {
                do {
                ch = fgetc(f1);
                printf("%c", ch);
                } while (ch != ' ');
                printf(" is an argument\n");
                i = -1;
                ar++;
                count++;
                break;
        }
        if (ch == ',' || ch == ';' || ch == '(' || ch == ')' || ch == '{' || ch == '}' || ch == '[' || ch ==
        ']') {
                printf("%c is a separator\n", ch);
                sp++;
                count++;
                str[i] = '\0';
                keywI(str);
                i = -1;
                break;
        }
        }
}
if (i != -1) {
        str[i] = ch;
        i++;
} else {
        i = 0;
}
```

```
}
printf("\nKeywords: %d\nIdentifiers: %d\nOperators: %d\nNumbers: %d\nSeparators:
%d\nArguments: %d\n", kw, id, op, num, sp, ar);
printf("Total number of tokens: %d\n", count);
printf("Number of lines: %d\n", new_line);
fclose(f1);
return 0;
}
void keywI(char *p) {
int k, flag = 0;
for (k = 0; k < 32; k++) {
if (strcmp(keys[k], p) == 0) {
        printf("%s is a keyword\n", p);
        kw++;
        count++;
        flag = 1;
        break;
}
}
if (!flag) {
if (isdigit(p[0])) {
        printf("%s is a number\n", p);
        num++;
        count++;
} else {
        if (p[0] != '\0') {
        printf("%s is an identifier\n", p);
        id++;
        count++;
        }
}
}
i = -1;
}
```

Laxcode.txt:

```
Sum = a + b

Sub = a - b
```

OUTPUT:

MARPU KARTHIK

AI NEW C RUN

STDIN

Input for the program (Optional)

Output:

```
= is an operator
sum  is an identifier
+ is an operator
 a  is an identifier
= is an operator
 b
sub  is an identifier
- is an operator
 a  is an identifier

Keywords: 0
Identifiers: 4
Operators: 4
Numbers: 0
Separators: 0
Arguments: 0
Total number of tokens: 8
Number of lines: 1
```

Practical - 02

Aim: program to count digits, vowel and symbols in C.

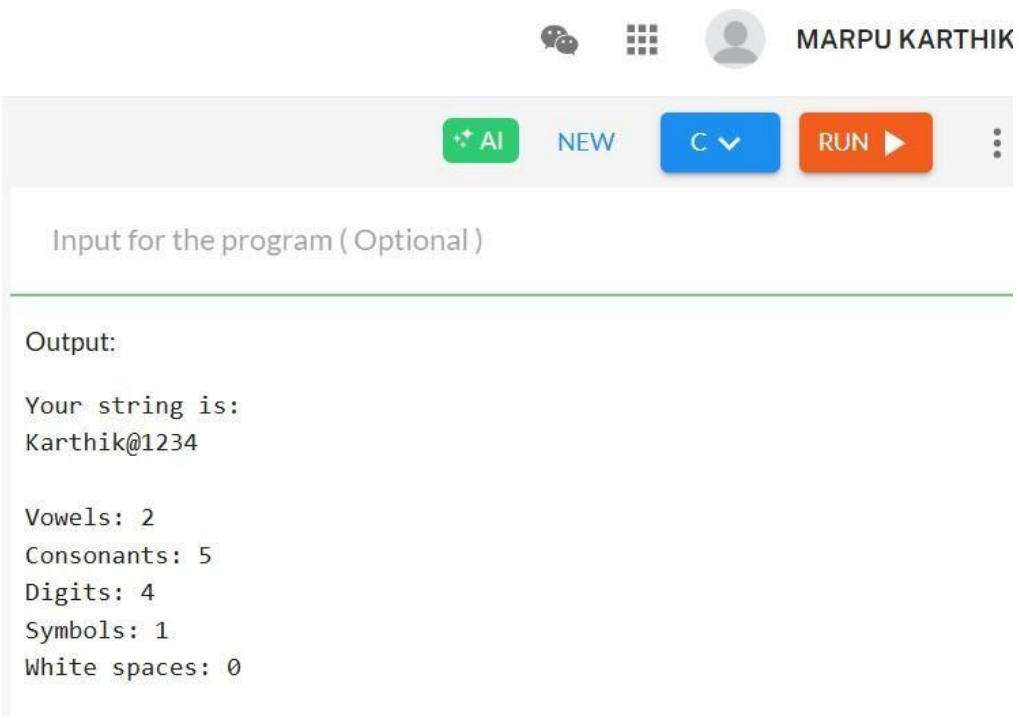
Program:

```
#include  <stdio.h>

#include <string.h>

#include <stdlib.h>

int main() {

    char str[1000];

    int i = 0;

    int vowels = 0, consonants = 0, digits = 0, symbols = 0, spaces = 0;

    FILE *fp;

    char ch;

    fp = fopen("input.txt", "r");

    if (fp == NULL) {

        printf("File not found\n");

        exit(1);

    }

    while ((ch = fgetc(fp)) != EOF) {

        str[i++] = ch;

    }

    str[i] = '\0';

    fclose(fp);

    printf("Your string is:\n%s\n", str);
```

```
    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] == 'a' || str[i] == 'A' || str[i] == 'e' || str[i] == 'E' ||
            str[i] == 'i' || str[i] == 'I' || str[i] == 'o' || str[i] == 'O' ||
            str[i] == 'u' || str[i] == 'U') {
            vowels++;
        } else if ((str[i] >= 'a' && str[i] <= 'z') || (str[i] >= 'A' && str[i] <= 'Z')) {
            consonants++;
        } else if (str[i] >= '0' && str[i] <= '9') {
            digits++;
        } else if (str[i] == ' ') {
            spaces++;
        } else {
            symbols++;
        }
    }
    printf("\nVowels: %d", vowels);
    printf("\nConsonants: %d", consonants);
    printf("\nDigits: %d", digits);
    printf("\nSymbols: %d", symbols);
    printf("\nWhite spaces: %d\n", spaces);
    return 0;
}
```

Input.txt: Karthik@1234

OUTPUT:


MARPU KARTHIK
AI
NEW
C
RUN
Input for the program (Optional)
Output:
Your string is:
Karthik@1234
Vowels: 2
Consonants: 5
Digits: 4
Symbols: 1
White spaces: 0

Practical - 03

AIM: Program to check validation of User Name and Password in C.

Program:

```
#include<stdio.h>

#include<string.h>

#include<stdlib.h>


int main(){

    char str[25], pwd[20];

    int a, i = 0, j = 0, h = 0, m = 0;

    FILE *fp, *fp1;

    char ch, ch1;


    fp = fopen("user.txt", "r");

    if (fp == NULL) {

        printf("File not opened ");

        exit(1);

    }


    ch = fgetc(fp);

    printf("Your Username is :\n");

    while (!feof(fp)) {

        str[i++] = ch;
```

```
        ch = fgetc(fp);
    }
    str[i] = '\0';
    printf("%s", str);
    fclose(fp);

    if (strcmp(str, "JayDepani") == 0 || strcmp(str, "DepaniJay") == 0 || strcmp(str, "JayPatel")
== 0 || strcmp(str, "PatelJay") == 0) {
        printf("\n This Username is Already there, please try to other Username.");
        return 0;
    }

    if (str[0] < 'A' || str[0] > 'Z') {
        printf("\n First character of the Username should be Capital.");
    }

    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] >= '0' && str[i] <= '9') {
            m = 1;
            break;
        }
        m = 0;
    }

    if (m == 1) {
        printf("\n Digit is not Allow in the Username.");
```

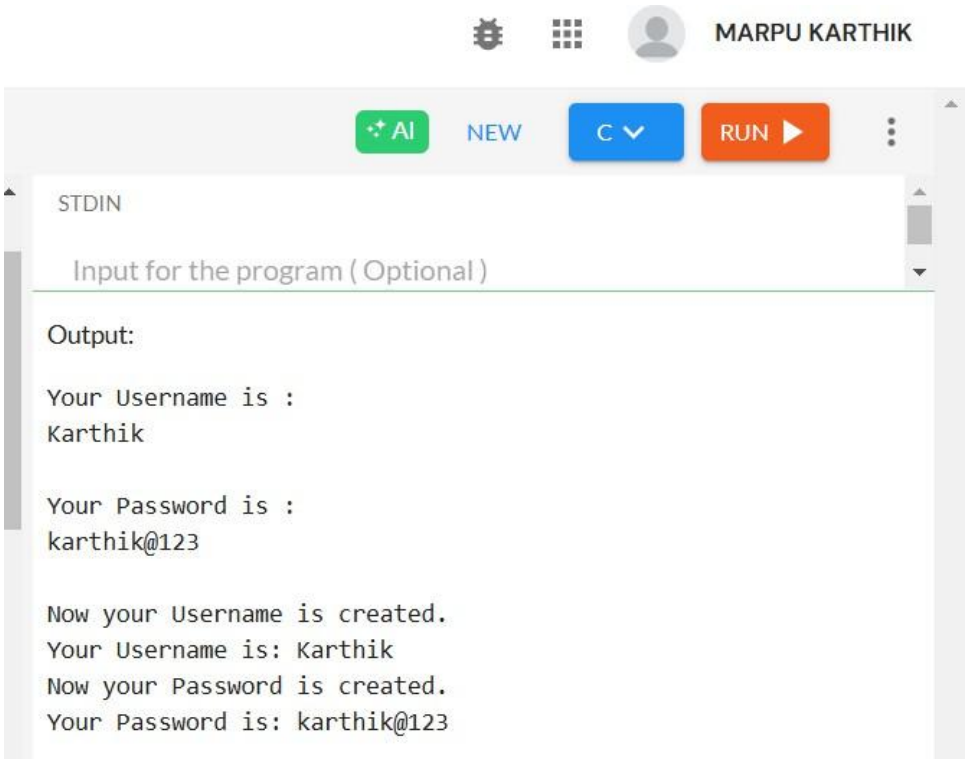
```
    }

    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] == '~' || str[i] == '!' || str[i] == '@' || str[i] == '#' || str[i] == '$' || str[i] == '%' ||
str[i] == '^' || str[i] == '&' || str[i] == '*') {
            m = 1;
            break;
        }
        m = 0;
    }

    if (m == 1) {
        printf("\n Spacial Symbols are not Allow in Username.");
        return 0;
    }

    fp1 = fopen("pass.txt", "r");
    if (fp1 == NULL) {
        printf("File not opened ");
        exit(1);
    }

    ch1 = fgetc(fp1);
    printf("\nYour Password is :\n");
    while (!feof(fp1)) {
        pwd[j++] = ch1;
```

```
    ch1 = fgetc(fp1);
}
pwd[j] = '\0';
printf("%s", pwd);
fclose(fp1);

a = strlen(pwd);
if (a < 8 || a > 15) {
    printf("\n Password length should be 8 to 15 characters.");
}

for (i = 0; pwd[i] != '\0'; i++) {
    if (pwd[i] >= '0' && pwd[i] <= '9') {
        h = 0;
        break;
    }
    h = 1;
}

if (h == 1) {
    printf("\n Minimum one digit is complasary to use in password like 1,2,3...");
}

for (i = 0; pwd[i] != '\0'; i++) {
    if (pwd[i] == '~' || pwd[i] == '!' || pwd[i] == '@' || pwd[i] == '#' || pwd[i] == '$' || pwd[i]
== '%' || pwd[i] == '^' || pwd[i] == '&' || pwd[i] == '*') {
```

```
        h = 0;
        break;
    }
    h = 1;
}

if (h == 1) {
    printf("\n Must be use one Spacial Symbols in Password like ~,!,@,#,$,%,^,&,*,.");
    return 0;
}

for (i = 0; pwd[i] != '\0'; i++) {
    if (pwd[i] == ' ') {
        h = 1;
        break;
    }
    h = 0;
}

if (h == 1) {
    printf("\n Password should not contain any space.");
    return 0;
}

printf("\n Now Your Username is Created.\n Your Username is:%s", str);
```

```
    printf("\n Now your Password is Created.\n your Password is :%s", pwd);
return 0;
}
```

User.txt: Compiler

Pass.txt: Compiler@123

OUTPUT: